Reproducing StreamingLLM: Attention Sinks for Infinite-Length Streaming Inference on a Pretrained Llama-Family Model

[Your Name]
[Department, Institution]
[email]

***Abstract*—Large language models cache the key and value (KV) states of every previous token, which makes long-running streaming use expensive, and they degrade once the input exceeds their pre-training length. Keeping only the most recent tokens (window attention) bounds memory but fails as soon as the first tokens are evicted. Xiao et al. [1] explain this with the *attention sink* effect and propose StreamingLLM: keep a few initial tokens permanently, plus a rolling window of recent tokens. I re-implement this method from scratch in TensorFlow on top of the pretrained TinyLlama-1.1B weights (a Llama-2-architecture model), including RoPE, grouped-query attention and a rolling KV cache with position re-assignment. On a 6,000-token stream from PG-19, perplexity is 8.39 for StreamingLLM (4+1020 cache), against 164.4 for window attention (0+1024) and 664.0 for dense attention. Attention analysis confirms that more than half of the attention mass lands on the four sink slots in most layers. I further ablate [summarise your ablation result in one sentence once you have run it].**

***Index Terms*—attention sink, streaming inference, KV cache, rotary position embedding, length generalization**

I. Problem and Motivation

Applications such as multi-round dialogue should run indefinitely, but a decoder-only transformer has two obstacles in that setting [1]. First, the KV cache grows linearly with the number of processed tokens, so memory and per-token latency grow without bound. Second, a model pre-trained on sequences of length L (2048 for TinyLlama [2], 4096 for Llama-2 [3]) performs poorly on positions beyond L.

The obvious fix is *window attention* [4]: keep only the last w KV pairs. Memory is then constant, but quality collapses as soon as the stream becomes longer than w, i.e. exactly when the very first token is dropped. A second baseline, sliding-window re-computation, avoids the collapse but re-runs a quadratic-cost forward pass for every new token. The paper's goal is a method that is as cheap as window attention and as accurate as re-computation, *without any fine-tuning*. This matters for me practically because the assignment uses an already pre-trained model, so the learned "sink token" variant of the paper (which needs pre-training) is not available; I use the training-free variant with the first four tokens as sinks.

II. Method

A. Why window attention fails: attention sinks

For a query at position t with scores $x_j = q_t^\top k_j/\sqrt{d}$, softmax forces the weights to sum to one:

$$\alpha_j = \frac{e^{x_j}}{\sum_{i\in\mathcal{C}} e^{x_i}}, \qquad \sum_{j\in\mathcal{C}} \alpha_j = 1, \tag{1}$$

where $\mathcal{C}$ is the set of cached tokens. When the query has no strongly relevant key, the model still has to put its probability mass somewhere. Trained autoregressive models learn to dump this surplus on the first few tokens, which are visible to every later token and are therefore the easiest to turn into a shared "sink" [1]. If a sink has a large logit $x_1 \gg x_j$, it contributes a large part of the denominator in (1). Evicting it removes that part, the remaining weights are renormalised to very different values, and the layer output is pushed out of distribution. This also explains why the failure is independent of the *content* of the evicted tokens.

B. StreamingLLM cache

StreamingLLM splits the cache $\mathcal{C}$ into s sink tokens and the w most recent tokens:

$$\mathcal{C}_t = \{0,\dots,s{-}1\} \cup \{t{-}w{+}1,\dots,t\}, \qquad |\mathcal{C}_t| = s{+}w = B, \tag{2}$$

so memory is constant at B entries per layer and head. With $s = 4$ and $B = 1024$ I write the configuration as $4{+}1020$.

C. Positions inside the cache

Llama-family models use rotary embeddings (RoPE) [5], which rotate each 2-D pair of query/key channels by an angle proportional to the position, so that the score depends only on the relative offset:

$$\big(R_m q\big)^\top \big(R_n k\big) = q^\top R_{n-m}\, k. \tag{3}$$

Once tokens have been evicted, the cached tokens have gaps in their original positions (for instance $\{0, 1, 2, 3, 996, \dots\}$). StreamingLLM assigns positions by *slot in the cache*, $\pi(j) = j$ for $j = 0,\dots,B{-}1$, not by original text position, so the largest relative distance the model ever sees is $B{-}1 < L$. To make this possible, keys are cached *before* rotation and all cached keys are re-rotated by their current slot at every step.

TABLE I

PERPLEXITY ON THE FIRST PG-19 TEST BOOK. OURS: TINYLLAMA-1.1B, FIRST 6K TOKENS, B=1024. PAPER: LLAMA-2-13B, FULL 65K TOKENS (ITS TABLE 1). MODELS AND STREAM LENGTHS DIFFER, SO ONLY THE *ordering and the recovery* ARE COMPARABLE, NOT THE ABSOLUTE VALUES.

Method	Ours	Ours (>1024)	Paper [1]
Dense (full cache)	664.0	1673.6	fails $> L$
Window 0+1024	164.4	311.0	5158.07
StreamingLLM 4+1020	**8.39**	**8.6**	5.40

III. IMPLEMENTATION DETAILS

Model and weights. HuggingFace has no TensorFlow Llama, so I download the `safetensors` weights of TinyLlama-1.1B (22 layers, 32 query heads, 4 key/value heads, head dimension 64, θ=10^4, pre-training length 2048) and write the forward pass myself: RMSNorm computed in fp32, RoPE with the split-half `rotate_half` convention, grouped-query attention via `repeat_kv`, SwiGLU MLP, and tied-or-separate LM head. Weights are fp16 on a Tesla T4.

Two code paths. `forward_full` processes a whole sequence with a causal mask (used for the attention maps and as a reference). `step` processes one token using a `RollingKVCache` that stores un-rotated keys and values per layer and evicts the middle once the cache holds more than $s+w$ entries. The cache logic keeps the first s entries and the last w entries; a unit test with s=2, w=3 over eight tokens yields tokens $[0, 1, 5, 6, 7]$ as expected.

Correctness checks. (i) A RoPE test shows that $q^\top k$ is identical for equal offsets (12.1778 at offsets $(10, 3)$, $(1010, 1003)$ and $(3000, 2993)$) and different for a different offset (5 instead of 7: 4.375), verifying (3). (ii) With the cache set to dense (no eviction), `step` and `forward_full` agree to a maximum absolute logit difference of 0.055 over 48 tokens; I attribute this to fp16 accumulation order, but **[re-run once in fp32 to confirm it shrinks]**.

Evaluation protocol. Data: the first book of the PG-19 test set [6] (65,134 Llama tokens); I evaluate the first 6,000 tokens, one token at a time, and report exp of the mean next-token cross-entropy. Following the paper's practice of using half the pre-training window, the budget is B=1024 for TinyLlama (pre-training length 2048). Compared methods: dense (unbounded cache), window (0+1024), StreamingLLM (4+1020).

IV. RESULTS VERSUS THE PAPER

Fig. 1 and Table I show the main result. The three methods are identical until 1024 tokens (running perplexity 7.59 at token 1000), which also confirms that the cache, re-rotation and masking are consistent when nothing has been evicted. After the cache fills, window attention jumps to a local perplexity of several hundred. Dense attention is fine until about the pre-training length of 2048 and then diverges, reaching local perplexities above 10^4 by about token 2,500. StreamingLLM stays flat at roughly 6–13 for the full stream.

Agreement with the paper. (1) Window attention breaks exactly when the first tokens are evicted. (2) Adding four sink tokens restores perplexity to the level of the unevicted prefix (7.6 at 1000 tokens vs. 8.4 over 6000), the same qualitative recovery as 5158→5.40 in the paper. (3) Dense attention fails beyond the pre-training length, as in the paper's Fig. 3.

Differences. My window-attention degradation (164, or 311 after token 1024) is far milder than the paper's 5158. I did not test why; plausible but unverified reasons are a different (smaller, GQA) model, a much shorter stream, and a different cache budget relative to the pre-training length. I also did not implement the sliding-window-with-re-computation oracle that the paper uses to show StreamingLLM matches the best-case quality, so I cannot make that claim for my setup.

Attention analysis. On a 128-token prefix, the mean attention to token 0 is about 0.7–0.9 in layers 3–8 and 0.4–0.6 in later layers, but near zero in layers 0–2 (the lower layers behave locally), matching the paper's observation that sinks appear beyond the bottom two layers. Fig. 2 shows the same effect inside the rolling cache: StreamingLLM keeps the sink mass, whereas window attention loses it, which is the mechanism behind the perplexity gap.

V. ABLATION

The paper ablates the number of initial tokens (its Table 2), the cache size (Table 6), and whether the sink content matters (Table 1). I run three ablations on the same TinyLlama setup, with a 4,000-token stream and total cache B=1024 unless stated. All variants use the same code path and differ in one factor only.

A. Number of sink tokens $s \in \{0, 1, 2, 4, 8, 16\}$ with $s + w = 1024$ fixed. *Question:* are four sinks really needed for a model that was not trained with a dedicated sink token? The paper reports, for Llama-2-7B, perplexity 3359.95 / 11.88 / 10.51 / 9.59 / 9.54 for $s = 0/1/2/4/8$ (its Table 2).

B. Position assignment: RoPE position = cache slot (StreamingLLM) versus original text position. *Question:* does the in-cache re-assignment of Sec. II-C matter? The paper states that it is crucial but, to my knowledge, does not tabulate this comparison.

C. Sink content: replace the first four tokens of the stream by the newline token (the paper does this in its Table 1: 5.40 vs. 5.60 on Llama-2-13B). *Question:* do the sinks work because of their position or because of their meaning?

Discussion. **[Write 1–2 paragraphs after you have the numbers: (1) at which s does perplexity stop improving, and does this match the paper's "four suffice"? (2) Does original-position assignment fail, and does the failure start near 1024 or near 2048 (the pre-training length)? (3) Is the newline variant close to the original (position matters) or not (content matters)? Report what you actually observe, including surprises; do not copy the paper's expected outcome.]**

VI. FAILURE CASES AND LIMITATIONS

Dense attention collapses beyond the training length. Local perplexity starts rising near token 2,200 and exceeds

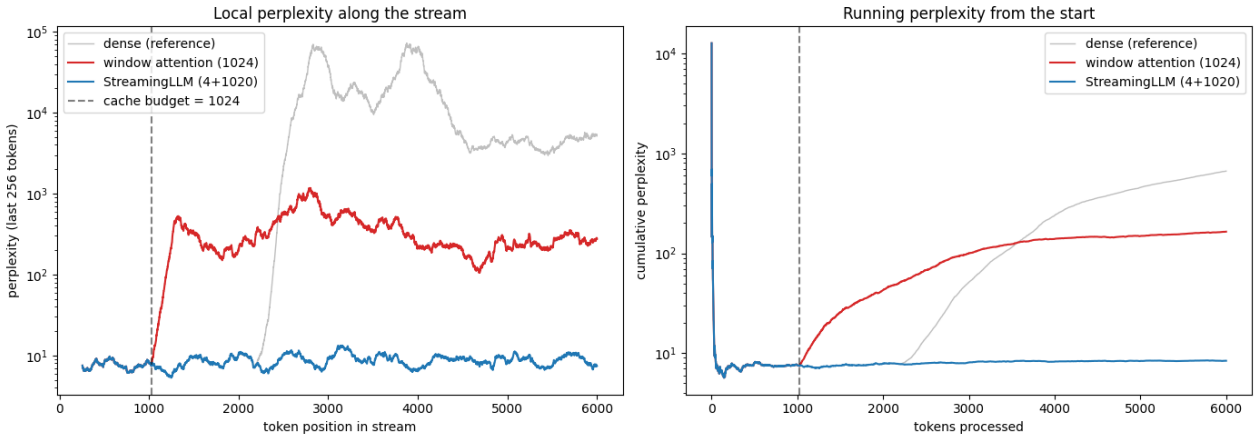


Fig. 1. Perplexity along a 6,000-token PG-19 stream with TinyLlama-1.1B. Left: perplexity of the last 256 tokens. Right: cumulative perplexity. Dashed line: cache budget (1024); StreamingLLM keeps 4 sinks + 1020 recent tokens.

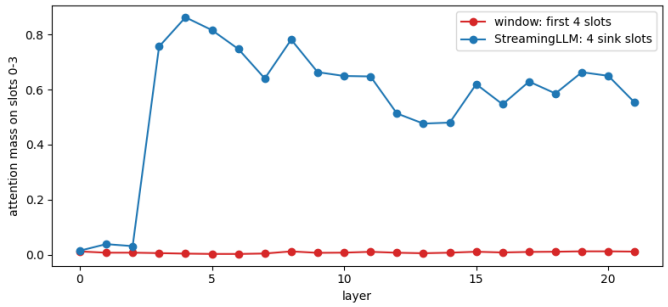


Fig. 2. Average attention mass on cache slots 0–3 per layer (cache size 256, 600-token stream). With sinks kept (blue), layers 3 and above put roughly 0.5–0.85 of their attention on the four sinks; with plain window attention (red) the first four slots hold ordinary recent tokens and receive about 0.01.

TABLE II
ABLATION RESULTS (TINYLLAMA-1.1B, PG-19 BOOK 1, 4,000 TOKENS, B=1024). [FILL FROM ABLATION_RESULTS.CSV PRODUCED BY THE NOTEBOOK CELLS.]

	Config	PPL (all)	PPL (>1024)
A	0+1024	[–]	[–]
A	1+1023	[–]	[–]
A	2+1022	[–]	[–]
A	4+1020	[–]	[–]
A	8+1016	[–]	[–]
A	16+1008	[–]	[–]
B	4+1020, slot pos.	[–]	[–]
B	4+1020, orig. pos.	[–]	[–]
C	4+1020, real first 4	[–]	[–]
C	4+1020, first 4 = newline	[–]	[–]

10^4 by about token 2,500, so unbounded caching is not even a quality-preserving baseline past L.

Window attention only partly recovers. After the first eviction it stays in the hundreds (311 on average past token 1024) and never returns to the dense level, because the sinks are gone for good.

Sinks are absent in the lowest layers. In layers 0–2 almost no mass goes to the first tokens (Fig. 2), so these layers are not helped by the sink mechanism; they rely on local context.

Perplexity does not measure memory. StreamingLLM remembers only what is inside the cache; information that was evicted is gone [1]. My language-modelling perplexity cannot show this, and I did not run a retrieval-style test (such as the paper's StreamEval), so I make no claim about long-range recall.

Numerical and scope limits. The 0.055 maximum logit gap between the cached and the full forward pass was only checked in fp16. The evaluation uses one book, 6K tokens (4K for ablations), one model and one cache budget, whereas the paper evaluates 4M-token streams on several model families. I did not measure latency or memory, and my per-token Python loop re-rotates the whole cache at every layer, so the paper's speed-up over re-computation is not reproduced here.

What did not work / what I changed. [Add your own honest notes from development here (bugs you hit, things you tried that failed, e.g. wrong RoPE convention, forgetting to cache un-rotated keys, first run of the logit-equivalence test failing). Examiners value this section; only include things that really happened.]

VII. CONCLUSION

A training-free change to the KV-cache eviction policy, keeping four initial tokens, turns a model that collapses at its cache size into one with flat perplexity on my 6K-token stream, and the attention maps show why. The result reproduces the qualitative claims of the paper on a smaller model; stronger claims (parity with re-computation, million-token streams, speed-ups) remain untested here.

REFERENCES

[1] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023, published at ICLR 2024.
[2] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.
[3] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.
[5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.
[6] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," in *International Conference on Learning Representations (ICLR)*, 2020.